

Agentic AI Workflows – Three More Live Experiments

Continuing the build log — Policy Compliance, Deep Research, and Visual QA — taken off the assignment brief and shipped as three working agents. Rule engines, planning loops, and multimodal retrieval, all grounded and inspectable.

06 · RULE-GROUNDED

Policy Compliance Agent

Synthetic data → rule engine → LLM judgment → compliance report

07 · PLAN-AND-REFLECT

Deep Research Agent Workflow

Planning, iterative research, drafting, and self-reflection

08 · MULTIMODAL

Image Retrieval / Visual QA System

Shared embedding space, retrieval, vision-grounded answering

Policy Compliance Agent

Rule-based evaluation over synthetic data

► generate → check → judge → report

Generates synthetic policy scenarios spanning clean, borderline, and violating cases, runs each through a deterministic rule engine first, then escalates anything ambiguous to an LLM judge that cites the exact rule it applied.

WORKFLOW

- 1

Synthetic case generation
An LLM generates realistic documents (emails, chat logs, support tickets) stratified across five policy categories, with a known ground-truth label for each.
- 2

Rule retrieval
The policy rulebook is embedded once; each case retrieves only the rules whose category matches its content, keeping the judge's context narrow.
- 3

Deterministic rule check
A first pass runs regex and keyword rules (SSNs, card numbers, blocklisted terms) — hard violations are flagged without ever calling the model.
- 4

LLM judgment
Cases the rule engine can't resolve outright go to an LLM judge, which returns a verdict, severity, and a citation to the specific retrieved rule.
- 5

Compliance report
Verdicts roll up into a report: violation counts by category, a severity breakdown, and every flagged case linked back to its rule.

POLICY RULEBOOK

- pii_exposure

SSNs, card numbers, personal emails shared outside approved channels
- confidentiality

Internal-only labeled content sent to external recipients
- financial_disclosure

Unverified guarantees, material non-public information
- conduct

Hostile, harassing, or discriminatory language
- regulatory_claims

Unsubstantiated health, safety, or legal claims

- KEY DETAILS

- Rule engine and LLM judge are two independent passes — a case only reaches the model once deterministic checks can't resolve it, keeping hard violations cheap and fast.
 - Every verdict cites the exact rule text it applied, so the report is auditable rather than a bare pass/fail.

IMPLEMENTATION

```
def rule_check(text: str) -> list:
    # deterministic pass: regex + keyword rules, no LLM call
    hits = []
    for rule in DETERMINISTIC_RULES:
        if rule.pattern.search(text):
            hits.append(rule.id)
    return hits

def judge_case(text: str, rules: list) -> dict:
    rule_text = "\n".join(f"- {r.id}: {r.text}" for r in rules)
    prompt = f"""Evaluate this case against the rules below.
Return: verdict, severity (low/med/high), rule_cited, rationale.

Rules:
{rule_text}"""
    return parse_verdict(call_llm(prompt, text))

def evaluate_document(text: str) -> dict:
    hard_hits = rule_check(text)
    if hard_hits:
        return {"verdict": "violation", "rule_cited": hard_hits, "source": "rule_engine"}
    relevant = retrieve_rules(text, top_k=3)
    return judge_case(text, relevant)
```

Deep Research Agent Workflow

Planning and reflection for grounded content generation

► plan → research → draft → reflect → revise

A single research pass misses things. This agent plans a set of sub-questions up front, researches each independently, drafts a grounded report, then reflects on its own coverage before revising — closing gaps instead of shipping them.

WORKFLOW

1 Planning

The LLM decomposes the research question into 3–6 sub-questions and a short outline, before any research begins.

2 Iterative research

Each sub-question runs its own search-and-summarize loop, producing structured findings with sources attached.

3 Draft synthesis

Findings are combined into a structured draft; every claim in the draft is required to trace back to a specific finding.

4 Reflection

A self-critique pass checks the draft against the original plan — unanswered sub-questions, thin source coverage, and unsupported claims are flagged.

5 Targeted revision

Flagged gaps trigger follow-up research on just those points; the draft is revised and re-checked, up to a capped number of loops.

RESEARCH CONTROLS

max_subquestions

3 / 5 / 6 — how many sub-questions the plan is allowed to spawn

max_reflection_loops

1 / 2 / 3 — how many revise-and-recheck cycles before finalizing

min_sources_per_subquestion

Floor on source diversity the reflection step checks against

KEY DETAILS

- Reflection is checked against the original plan, not just prose quality — so a hard sub-question can't be silently dropped from the final report.
- Revision is targeted: only the flagged sub-questions get re-researched, not the whole report from scratch.

IMPLEMENTATION

```
def plan_research(question: str, max_subq=5) -> list:
    prompt = f"Break this question into up to {max_subq} research sub-questions."
    return parse_list(call_llm(prompt, question))

def research_subquestion(subq: str, min_sources=3) -> dict:
    sources = web_search(subq)[:min_sources]
    summary = call_llm("Summarize these sources for the sub-question.", sources)
    return {"subquestion": subq, "summary": summary, "sources": sources}

def reflect_on_draft(draft: str, plan: list, findings: list) -> dict:
    prompt = f"""Check the draft against the plan.
    Flag: unanswered sub-questions, thin sources, unsupported claims."""
    return parse_reflection(call_llm(prompt, draft))

def run_research_agent(question: str, max_loops=2) -> str:
    plan = plan_research(question)
    findings = [research_subquestion(q) for q in plan]
    draft = draft_report(findings)
    for _ in range(max_loops):
        gaps = reflect_on_draft(draft, plan, findings)
        if not gaps["issues"]:
            break
        findings += [research_subquestion(g) for g in gaps["missing"]]
        draft = draft_report(findings)
    return draft
```

Image Retrieval / Visual QA System

A multimodal retrieval and question-answering pipeline

► embed → index → retrieve → answer

Embeds images and captions into a shared multimodal vector space, retrieves the images closest to a question, and generates an answer grounded only in what was retrieved — with the option to say "not found" rather than guess.

WORKFLOW

1 Multimodal indexing

Each image is captioned and embedded with a CLIP-style encoder; the image and caption embeddings are stored together in one index.

2 Query embedding

The text question is embedded into the same shared space as the images, so text and image vectors are directly comparable.

3 Retrieval

Top-k images return by cosine similarity, combining signal from both the raw image embedding and its caption.

4 Grounded visual QA

A vision-capable model receives only the retrieved image(s) plus the question, and answers citing which image supports the answer.

RETRIEVAL SETTINGS

embedding_space

512-d / 768-d CLIP-style shared text-image encoder

top_k

1 / 3 / 5 — number of images passed to the answering step

grounding_mode

single-image answer, or multi-image synthesis across top-k

KEY DETAILS

- Retrieval matches on both caption text and raw image embeddings, catching visual similarity a caption-only search would miss.
- The QA step is instructed to return "not found in retrieved images" when nothing in the top-k actually supports the question, rather than answering from general knowledge.

IMPLEMENTATION

visual_qa.py

```
def index_image(image_path: str):
    caption = generate_caption(image_path)
    vector = clip_embed_image(image_path) # shared text-image space
    index.add(vector, meta={"path": image_path, "caption": caption})

def retrieve_images(question: str, top_k=3) -> list:
    query_vec = clip_embed_text(question)
    hits = index.search(query_vec, k=top_k, metric="cosine")
    return [h.meta for h in hits]

def answer_visual_question(question: str, top_k=3) -> str:
    images = retrieve_images(question, top_k)
    if not images:
        return "Not found in retrieved images."
    prompt = "Answer ONLY using the attached image(s). Cite which image you used."
    return call_vision_llm(prompt, question, images=[i["path"] for i in images])
```

Side-by-side

EXPERIMENT	DATA FED TO THE LLM	DISTINGUISHING FEATURE
Policy Compliance Agent	Case text + retrieved rules from a 5-category rulebook	Rule engine runs first; the model only judges what regex can't resolve
Deep Research Agent	Sub-question + summarized sources, then the full plan at reflection time	Reflection is checked against the original plan, not just prose quality
Image Retrieval / Visual QA	Question + top-k retrieved images from a shared CLIP-style index	Answers only from retrieved images; says "not found" instead of guessing

Common foundation

All three experiments are the same underlying move as the first three — ground a call in the right data, or split a task into simpler, checkable steps. The policy agent grounds judgment in retrieved rule text and backs it with a deterministic pass; the research agent grounds its draft in per-sub-question findings and checks itself against its own plan; the visual QA system grounds its answer in retrieved images rather than the model's general knowledge. What differs is not the model but what evidence reaches it, and how the agent checks its own work before returning an answer.